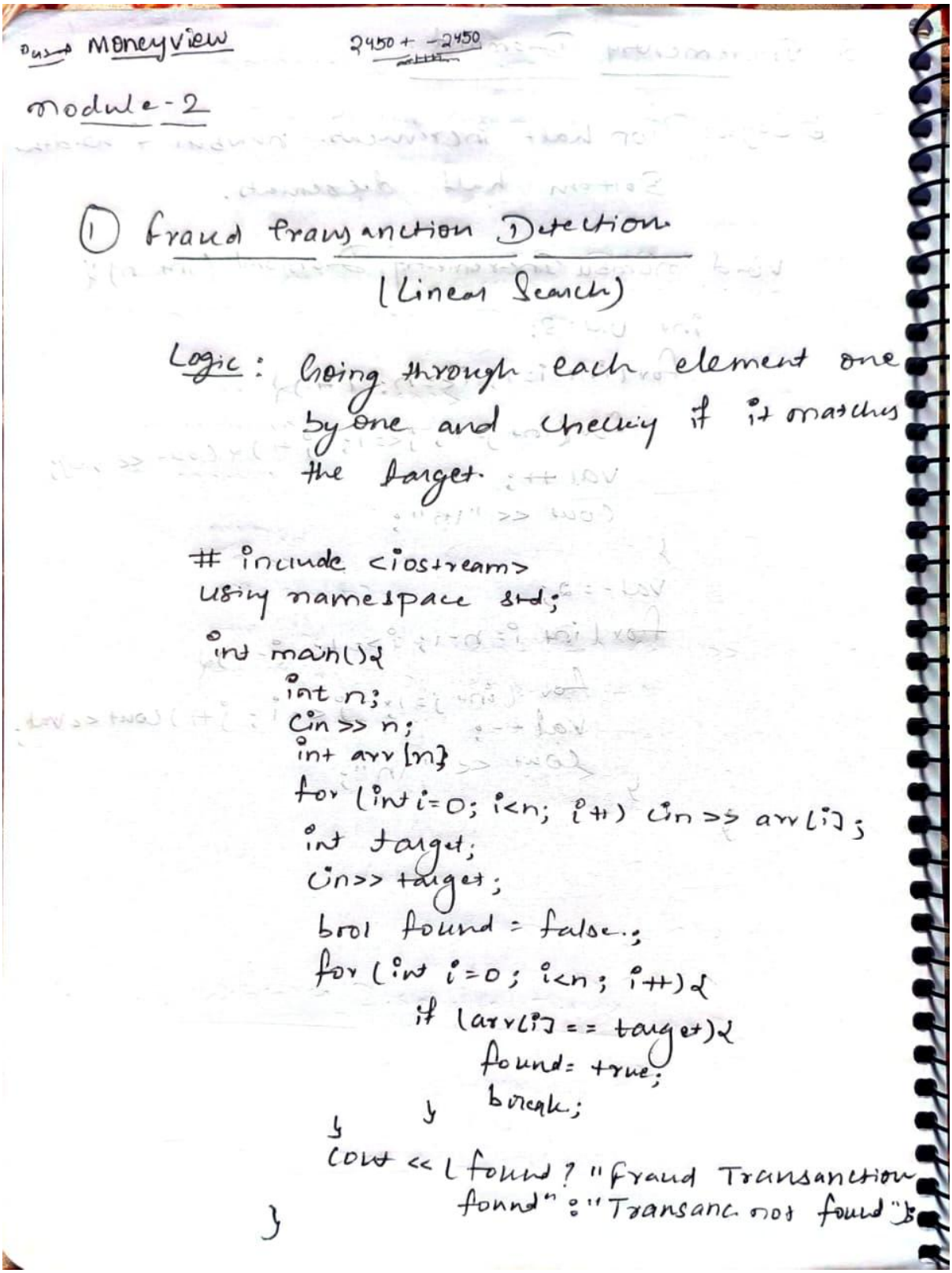
1. **Fraud Transaction Detection (Linear Search)**
   Problem Statement : A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists. Write a program using Linear Search.



Moneyview 2450 + -2450

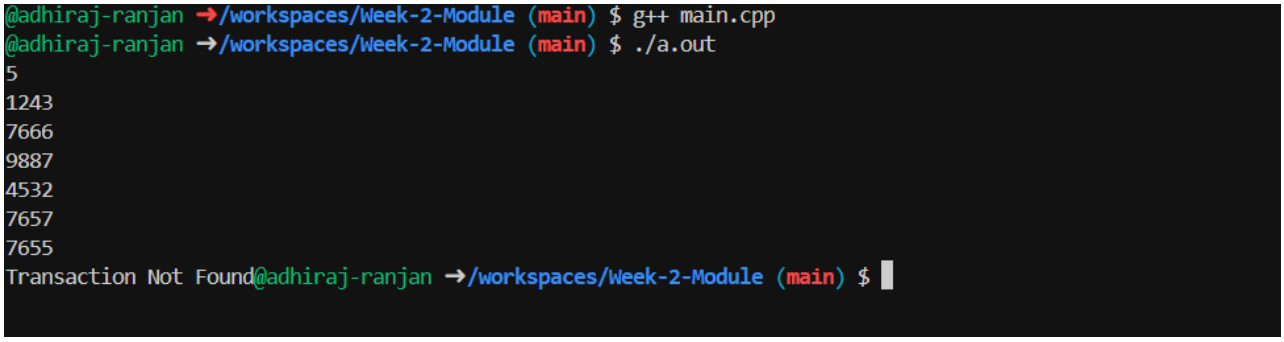
module-2

(1) Fraud transaction Detection
(Linear Search)

Logic : Going through each element one by one and checking if it matches the target.

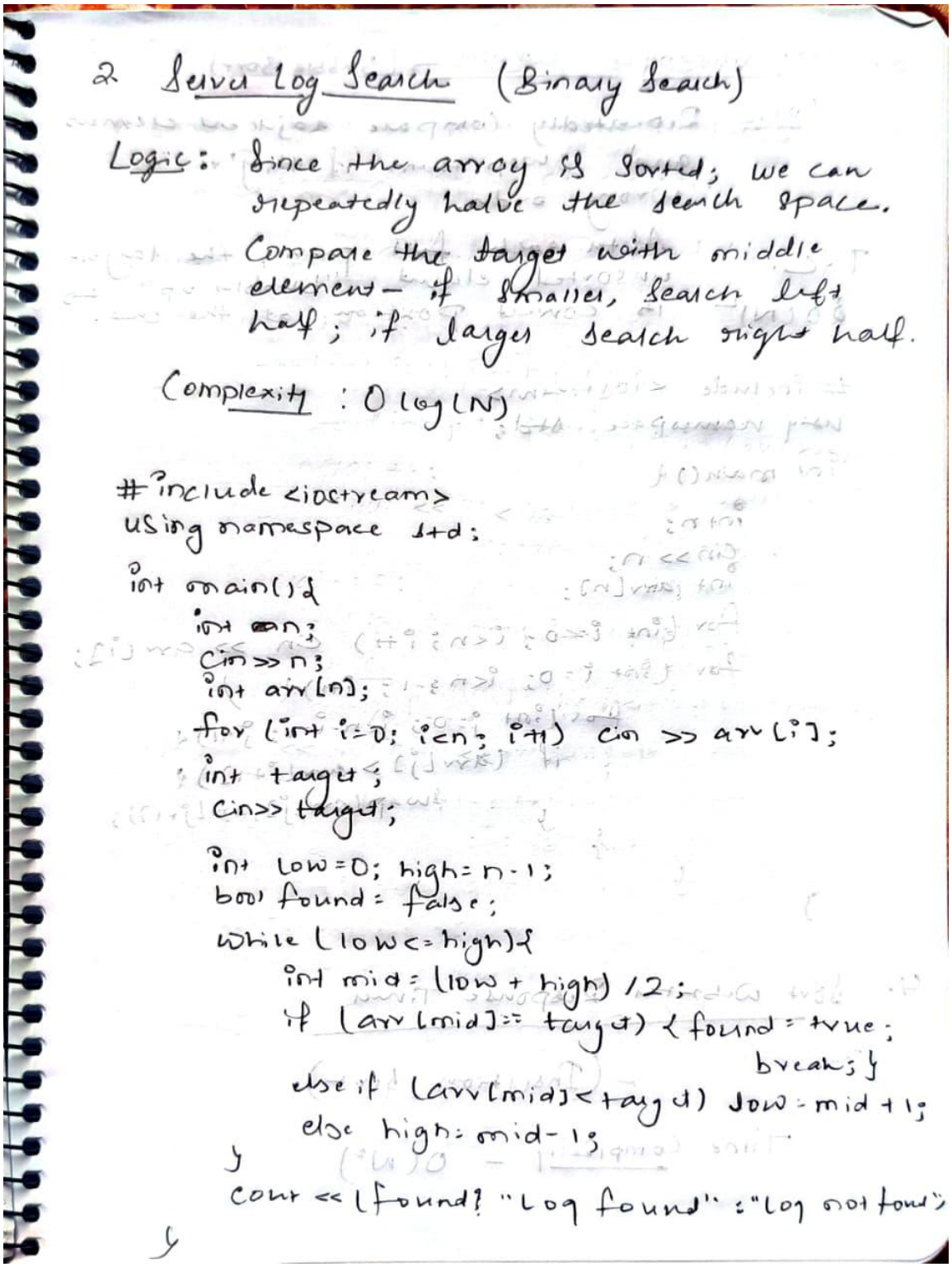
```
#include <iostream>
using namespace std;
int main(){
    int n;
    cin >> n;
    int arr[n]
    for (int i=0; i<n; i++) cin >> arr[i];
    int target;
    cin>> target;
    bool found = false;
    for (int i=0; i<n; i++){
        if (arr[i] == target){
            found= true;
            break;
        }
    }
    cout << (found ? "Fraud Transaction found" : "Transanc. not found");
}
```

Output :


@adhiraj-ranjan ➜/workspaces/Week-2-Module (main) $ g++ main.cpp
@adhiraj-ranjan ➜/workspaces/Week-2-Module (main) $ ./a.out
5
1243
7666
9887
4532
7657
7655
Transaction Not Found@adhiraj-ranjan ➜/workspaces/Week-2-Module (main) $

2. **Server Log Search (Binary Search)**

Problem Statement : A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists. Write a program using Binary Search.



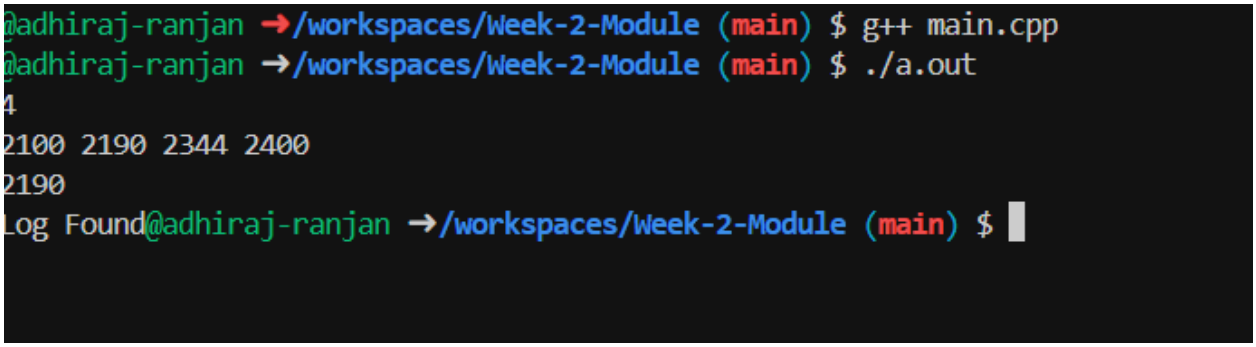
2 Server Log Search (Binary Search)

Logic: Since the array is sorted; we can repeatedly halve the search space. Compare the target with middle element – if smaller, search left half; if larger search right half.

Complexity : O log(N)

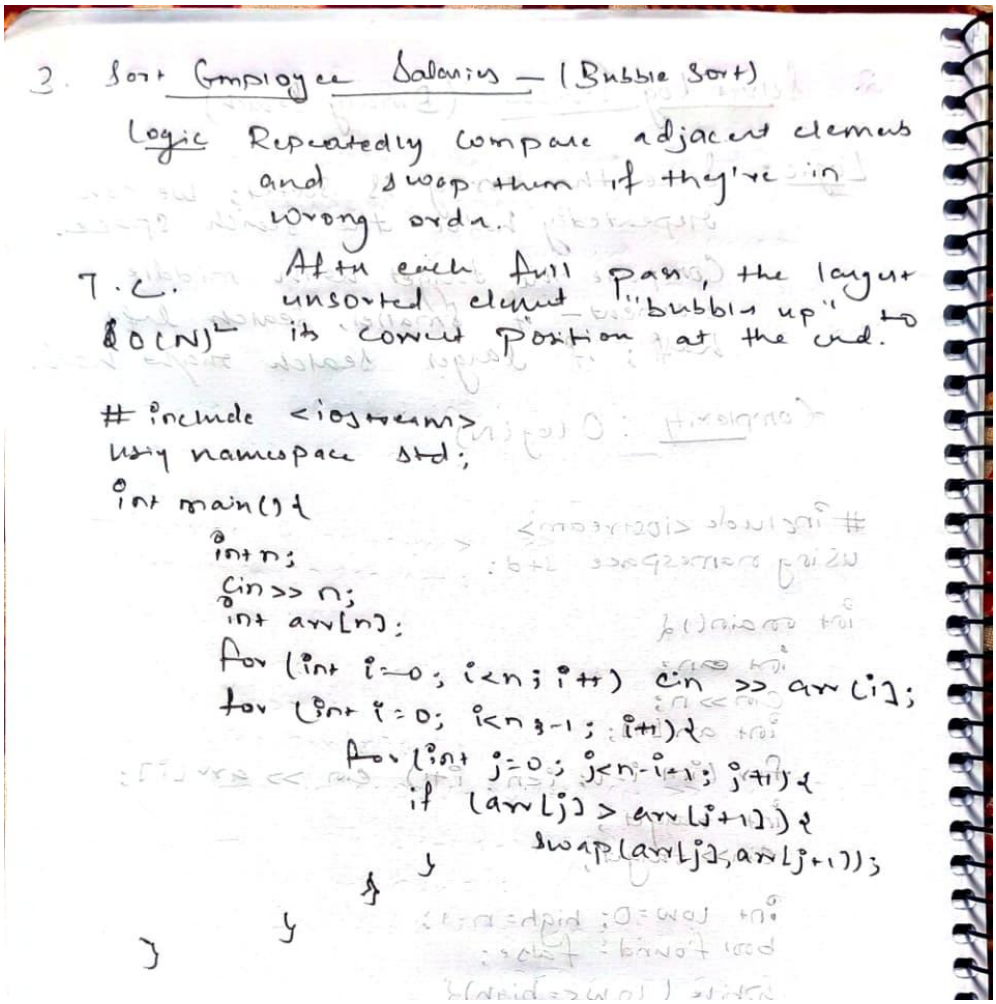
```
#include <iostream>
using namespace std;
int main(){
    int n;
    cin>>n;
    int arr[n];
    for(int i=0; i<n; i++) cin >> arr[i];
    int target;
    cin>> target;
    int low=0; high=n-1;
    bool found = false;
    while(low<=high){
        int mid = (low + high) /2;
        if (arr[mid]== target) { found = true;
                                 break;}
        else if (arr[mid]<target) low = mid +1;
        else high= mid-1;
    }
    cout<<(found? "Log found" :"Log not found");
}
```

Output :


@adhiraj-ranjan ➜/workspaces/Week-2-Module (main) $ g++ main.cpp
@adhiraj-ranjan ➜/workspaces/Week-2-Module (main) $ ./a.out
4
2100 2190 2344 2400
2190
Log Found@adhiraj-ranjan ➜/workspaces/Week-2-Module (main) $

3. **Sort Employee Salaries (Bubble Sort)**

Problem Statement : A company wants to sort employee salaries in ascending order for payroll analysis. Write a program using Bubble Sort.



2. Sort Employee Salaries — (Bubble Sort)

Logic: Repeatedly compare adjacent elements and swap them if they're in wrong order.

T.C. $O(N)^2$ — After each full pass, the largest unsorted element "bubbles up" to its correct position at the end.

```
#include <iostream>
using namespace std;
int main(){
    int n;
    cin >> n;
    int arr[n];
    for (int i=0; i<n; i++) cin >> arr[i];
    for (int i=0; i<n-1; i++){
        for (int j=0; j<n-i-1; j++){
            if (arr[j] > arr[j+1]){
                swap(arr[j], arr[j+1]);
            }
        }
    }
}
```

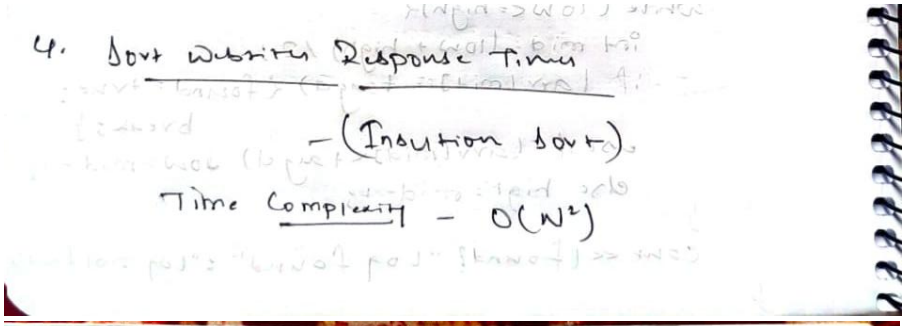
Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

@adhiraj-ranjan →/workspaces/Week-2-Module (main) $ g++ main.cpp
@adhiraj-ranjan →/workspaces/Week-2-Module (main) $ ./a.out
5
1833
3566
8222
2662
9294
1833 2662 3566 8222 9294@adhiraj-ranjan →/workspaces/Week-2-Module (main) $
```

4. **Sort Website Response Time (Insertion Sort)**

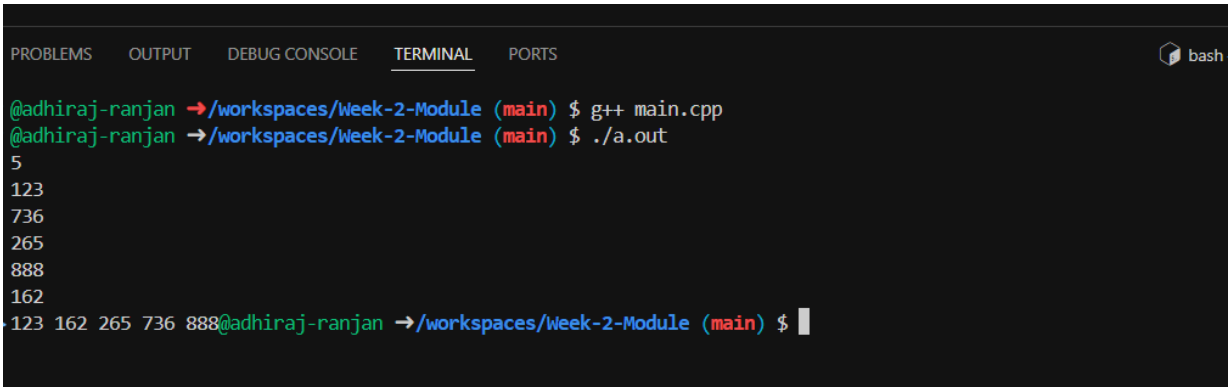
Problem Statement : A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position. Write a program using Insertion Sort



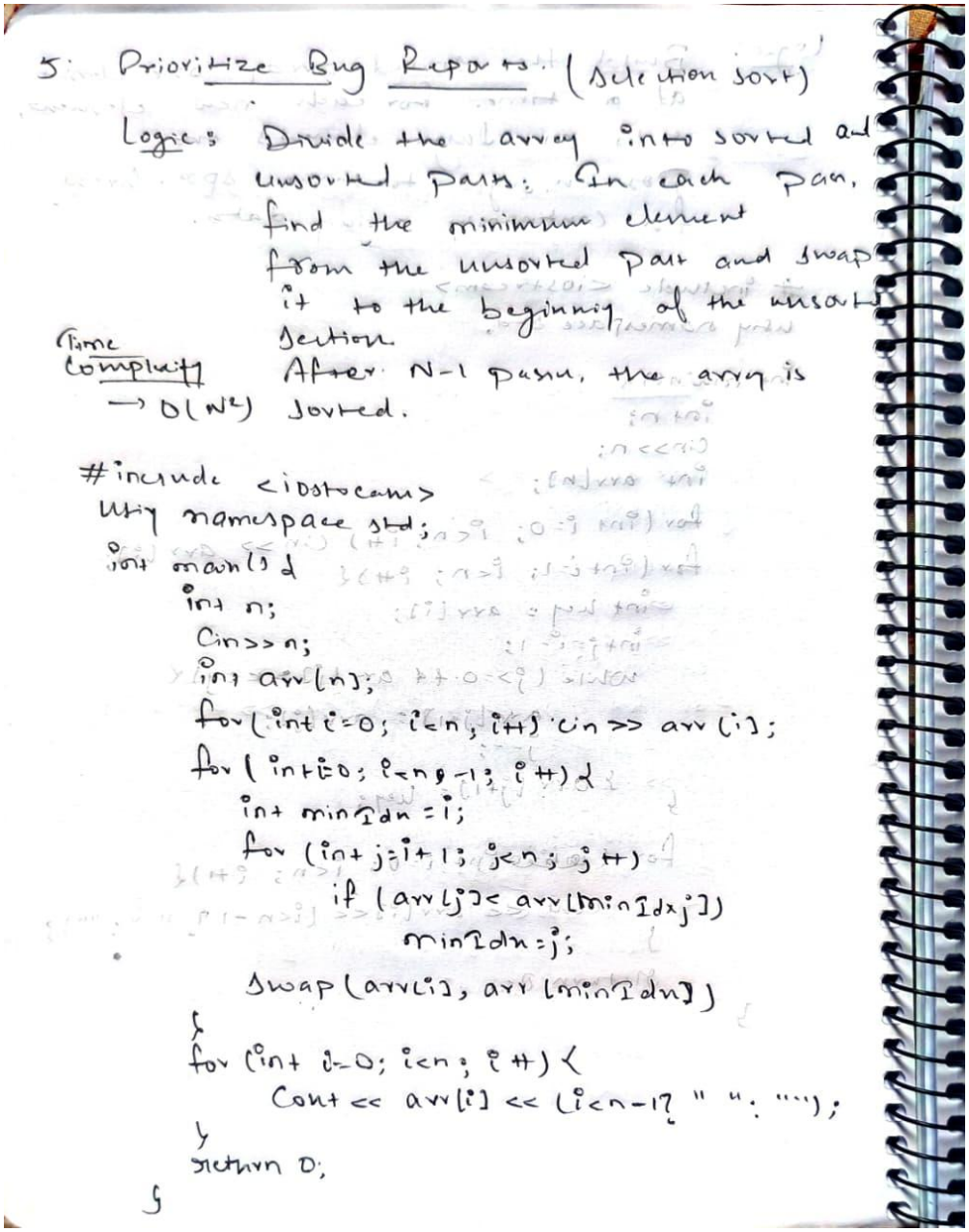
Logic: Build the sorted array one element at a time. For each new element, shift all larger elements one position right to make spot. Great for continuously arriving data.

```
#include <iostream>
using namespace std;
int main(){
    int n;
    cin>>n;
    int arr[n];
    for(int i=0; i<n; i++) cin>> arr[i];
    for(int i=1; i<n; i++){
        int key = arr[i];
        int j= i-1;
        while(j>=0 && arr[j]>key){
            arr[j+1] = arr[j];
            j--;
        } arr[j+1] = key;
    }
    for(int i=0; i<n; i++){
        cout<< arr[i]<< (i<n-1 ? " " : "");
    }
    return 0;
}
```

Output :



5. **Prioritize Bug Reports (Selection Sort)**
   Problem Statemen : A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order. Write a program using Selection Sort.



5: Prioritize Bug Reports (Selection sort)

Logic: Divide the array into sorted and unsorted parts. In each pass, find the minimum element from the unsorted part and swap it to the beginning of the unsorted section. After N-1 pass, the array is sorted.

Time Complexity → O(N²)

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin>>n;
    int arr[n];
    for(int i=0; i<n; i++) cin >> arr[i];
    for ( int i=0; i<n-1; i++) {
        int minIdx = i;
        for (int j=i+1; j<n; j++)
            if (arr[j]< arr[minIdx])
                minIdx=j;
        swap(arr[i], arr[minIdx])
    }
    for (int i=0; i<n; i++) {
        cout << arr[i] << (i<n-1? " ": "");
    }
    return 0;
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

@adhiraj-ranjan ➜/workspaces/Week-2-Module (main) $ g++ main.cpp
@adhiraj-ranjan ➜/workspaces/Week-2-Module (main) $ ./a.out
6
3 4 2 8 2 9
2 2 3 4 8 9@adhiraj-ranjan ➜/workspaces/Week-2-Module (main) $
```